

THROTTLING PLAN

APPLICATION RATE LIMITING

A targeted, fixed-window rate-limiting design for Cedric Masters Autos — written for review. This is a **proposal only**; no throttling code is implemented in the application today.

DOCUMENT ID CMA-TRP-001	STATUS PROPOSED — NOT IMPLEMENTED	DATE 07 Aug 2026	OWNER eng-team
APPROACH Fixed-window · DB counter	SURFACE Finance / dispatch / auth	EFFORT ~0.5 d	RISK Low · reversible

01

STATUS & POSITION

NOT YET IMPLEMENTED

This document is a **proposed throttling plan**. It has not been implemented in the application. git log shows no rate-limiting or throttling code in convex/ or src/. Adoption requires team sign-off and, per the recommendation below, should be limited to the surfaces that matter.

ENGINEERING POSITION (PUSHBACK)

Cedric Masters Autos is a **single-tenant staff application** deployed for one workshop's team. The typical threat model that motivates rate limiting — public signup, credential stuffing, scraping, bursty traffic from untrusted users — does not exist here: every caller is an authenticated employee with a role. Full-blown rate limiting would therefore be **over-engineering**. The honest recommendation is:

- **Do not** build a general-purpose rate limiter across every mutation.
- **Do** add two lightweight, high-value guards: (1) a soft cooldown on repeated financial mutations to absorb accidental double-clicks, and (2) limits on the auth/reset surface if it is ever exposed beyond the team.
- Revisit only when a **customer portal** or public booking endpoint ships (both are post-MVP per the PRD).

The plan below is written so the team can adopt it fully, partially, or defer it with confidence.

02 THREAT MODEL

SCENARIO	LIKELIHOOD	NOTES
Accidental double-submit on payments / dispatch	MODERATE	A stray double-click records two payments or dispatches twice; both are recoverable but create ledger noise.
Stray client retry loops on a flaky network	MODERATE	React Query retries can replay a mutation more than intended.
Credential stuffing on login/reset	LOW	Auth is staff-only, but the route is public by definition; guards there are cheap.
Scripted abuse / scraping	LOW	Only reachable with valid staff credentials — de-scoped for v1.
Public customer portal / booking	DEFERRED	Post-MVP; when it ships, throttling becomes mandatory, not optional.

CONCLUSION

The realistic, proportionate scope is **two narrow limits**: one per-user cooldown on financial mutations, one on the auth/reset flow. Everything else is deferred.

03

PROPOSED DESIGN

MECHANISM — FIXED-WINDOW COUNTER

- New table `rateLimits`: { `key`, `windowStart`, `count` }, indexed on `key`.
- Helper `enforceLimit(ctx, key, limit, windowMs)` called inside a mutation.
- Read the row; if the window elapsed, reset; else if `count ≥ limit` throw `ConvexError('Too many requests — try again shortly.')`; else increment.
- Atomic: Convex mutations run as single transactions, so check-and-increment cannot race.
- Key scope: per `userId` (never a bare IP).

HOUSEKEEPING

- Rows are reset in-place when a window expires — bounded size.
- Optional nightly cleanup removes rows older than 24 h.
- Feature-flag `RATE_LIMIT_ENABLED` (default `false`) so limits can be disabled instantly during demos without a deploy.
- Limits are per-environment config, not hard-coded business rules.

```
// convex/lib/rate-limit.ts — proposed
export async function enforceLimit(
  ctx: MutationCtx,
  key: string,
  limit: number,
  windowMs: number,
) {
  const now = Date.now()
  const row = await ctx.db.query('rateLimits').withIndex('key', q => q.eq('key', key)).first()
  if (!row || now - row.windowStart > windowMs) {
    if (row) await ctx.db.patch(row._id, { windowStart: now, count: 1 })
    else await ctx.db.insert('rateLimits', { key, windowStart: now, count: 1 })
    return
  }
  if (row.count >= limit) {
    throw new ConvexError('Too many requests. Please wait a moment and try again.')
  }
  await ctx.db.patch(row._id, { count: row.count + 1 })
}
```

04

PROPOSED LIMIT MATRIX

SURFACE	MUTATION	LIMIT	WINDOW	SCOPE	RATIONALE
Finance	payments.record	20	1 min	user	Absorb double-clicks; prevent ledger noise
Finance	invoices.generate / approve	30	1 min	user	Regenerate storms
Service	jobs.checkIn	40	1 min	user	Reasonable peak intake
Inventory	partsRequests.dispatch / reverse	40	1 min	user	Stock + invoice side effects are heavier
Sales	salesOrders.create / addPayment	30	1 min	user	Financial writes
Auth	sign-in attempts	10	5 min	email	Credential-stuffing guard (if exposed)
Auth	reset-password requests	5	1 h	email	Prevent inbox flooding
Data	parts.importParts	5	1 min	user	Bulk insert guard

Limits are intentionally generous for a small staff — they stop accidents and scripts, never a busy shift.

CLIENT-SIDE COMPLEMENT

BUTTON COOLDOWNS

Disable submit buttons while `isPending` (already the norm in this app) — the primary accidental-duplicate defence.

INPUT DEBOUNCE

Debounce customer/part/lead search to ~300 ms to avoid a query per keystroke (pairs with CR-06).

ERROR COPY

On a 429-style `ConvexError`, surface: "Too many requests — try again in a moment," never raw internals.

05

ADOPTION PATH & TESTING

STEP	ACTION	EFFORT
1	Add rateLimits table to convex/schema.ts (widen – additive, non-breaking)	15 min
2	Implement convex/lib/rate-limit.ts helper + RATE_LIMIT_ENABLED flag	30 min
3	Call enforceLimit at the top of the mutations in the matrix	20 min
4	Unit tests: window reset, limit hit, per-user isolation, flag off-path	30 min
5	Manual smoke: exceed a limit, confirm the friendly error and no side effects	15 min
6	Rollout: ship behind the flag, enable, watch for false positives in one shift	–

ROLLBACK

Flip RATE_LIMIT_ENABLED=false or redeploy the prior build – the table is additive and every limit is independent, so rollback is instant and non-destructive.